กรอบการทำงาน ReAct ในแบบจำลองภาษาขนาดใหญ่: การผสานพลังการใช้เหตุผลและการปฏิบัติเพื่อเสริมศักยภาพปัญญาประดิษฐ์

ส่วนที่ 1: ความรู้เบื้องต้นเกี่ยวกับกรอบการทำงาน ReAct ในแบบจำลองภาษาขนาดใหญ่

- **1.1. นิยาม ReAct: การบรรจบกันของการใช้เหตุผลและการปฏิบัติ**

  กรอบการทำงาน ReAct (Reasoning and Acting) เป็นกระบวนทัศน์ที่ช่วยให้แบบจำลองภาษาขนาดใหญ่ (Large Language Models หรือ LLMs) สามารถสร้างทั้งร่องรอยการใช้เหตุผลด้วยคำพูด (ความคิด) และการดำเนินการเฉพาะงาน (การปฏิบัติ) ในลักษณะที่สอดประสานกัน [1] การทำงานร่วมกันนี้ช่วยให้ LLMs สามารถใช้เหตุผลแบบพลวัตเพื่อสร้าง รักษา และปรับเปลี่ยนแผนการปฏิบัติงาน ขณะเดียวกันก็มีปฏิสัมพันธ์กับสภาพแวดล้อมภายนอกได้ [1] สิ่งนี้มีความสำคัญอย่างยิ่ง เนื่องจาก LLMs แบบดั้งเดิม แม้จะมีความสามารถในการสร้างข้อความที่น่าประทับใจ แต่บ่อยครั้งยังขาดความสามารถในการใช้เหตุผลที่แข็งแกร่ง หรือความสามารถในการโต้ตอบกับโลกภายนอกเพื่อตรวจสอบข้อเท็จจริงหรือดำเนินงานให้สำเร็จ [6] ReAct จึงมุ่งหวังที่จะลดช่องว่างดังกล่าว

  เป็นสิ่งสำคัญอย่างยิ่งที่จะต้องเน้นย้ำว่า ReAct ที่กล่าวถึงในรายงานนี้มีความเฉพาะเจาะจงกับ LLMs และแตกต่างจากไลบรารี JavaScript ที่ชื่อ "React" [7] รวมถึงกรอบการทำงานอื่นๆ ที่มีชื่อพ้องเสียงกัน เช่น "REACT" สำหรับการแก้ไของค์ความรู้ (knowledge editing) [12] การชี้แจงนี้มีความสำคัญอย่างยิ่งในการกำหนดบริบทที่ถูกต้องตั้งแต่เริ่มต้น

  การออกแบบกรอบการทำงาน ReAct ซึ่งผสมผสานการกระทำที่มุ่งเน้นงานเข้ากับการใช้เหตุผลด้วยคำพูด (หรือ "การพูดกับตัวเองภายในใจ") ได้รับแรงบันดาลใจอย่างชัดเจนจากกระบวนการแก้ปัญหาของมนุษย์ [1] มนุษย์มักจะอธิบายกระบวนการคิดของตนเองออกมาเป็นคำพูด โดยเฉพาะอย่างยิ่งเมื่อต้องรับมือกับงานที่ซับซ้อน เพื่อ

จัดโครงสร้างแนวทาง ติดตามความคืบหน้า และจัดการกับข้อยกเว้นที่อาจเกิดขึ้น [1] ReAct ได้นำหลักการนี้มาปรับใช้โดยการกระตุ้นให้ LLM สร้างขั้นตอน "ความคิด" (Thought) ที่ชัดเจน การเลียนแบบนี้ไม่ได้เป็นเพียงเรื่องผิวเผิน แต่มีจุดมุ่งหมายเพื่อให้ LLMs มีความสามารถในการแก้ปัญหาที่แข็งแกร่งและยืดหยุ่นมากขึ้น คล้ายกับวิธีที่มนุษย์เรียนรู้งานใหม่และปรับตัวเข้ากับความไม่แน่นอน [1] สิ่งนี้บ่งชี้ว่า ReAct ไม่ได้เป็นเพียงเทคนิคทางเทคนิคเท่านั้น แต่เป็นความพยายามที่จะสร้างระบบ AI ที่สามารถใช้เหตุผลและโต้ตอบกับโลกในลักษณะที่คล้ายคลึงกับมนุษย์มากขึ้น ซึ่งอาจนำไปสู่พฤติกรรมของ AI ที่เข้าใจง่ายและเป็นธรรมชาติมากขึ้น

- **1.2. จุดกำเนิดและความสำคัญของ ReAct ในการเสริมศักยภาพ LLM**

  ReAct ได้รับการนำเสนอครั้งแรกโดย Yao และคณะ ในบทความวิจัยปี 2023 เรื่อง "ReAct: Synergizing Reasoning and Acting in Language Models" [3] บทความนี้ได้วางรากฐานสำหรับแนวทางใหม่ในการสร้างตัวแทน (agents) ที่ใช้ LLM ซึ่งมีความสามารถและความน่าเชื่อถือสูงขึ้น แรงจูงใจเบื้องหลังการพัฒนา ReAct คือการจัดการกับข้อจำกัดของแนวทางก่อนหน้านี้ ตัวอย่างเช่น การกระตุ้นด้วยลูกโซ่แห่งความคิด (Chain-of-Thought หรือ CoT) แม้จะช่วยปรับปรุงการใช้เหตุผล แต่ก็ขาดการเข้าถึงข้อมูลภายนอกและอาจนำไปสู่การสร้างข้อมูลที่ไม่เป็นจริง (hallucination) และการแพร่กระจายของข้อผิดพลาด [1] ในขณะเดียวกัน แบบจำลองที่เน้นการปฏิบัติเพียงอย่างเดียว (action-only models) อาจขาดความสามารถในการใช้เหตุผลเพื่อแบ่งย่อยงานที่ซับซ้อนหรือปรับตัวเข้ากับสถานการณ์ใหม่ ๆ [4]

  ReAct นับเป็นก้าวสำคัญในการพัฒนากระบวนการทำงานแบบอัตโนมัติที่ขับเคลื่อนด้วย LLM (LLM-driven agentic workflows) [5] ช่วยให้ LLMs สามารถก้าวข้ามการสร้างข้อความธรรมดาไปสู่การแก้ปัญหาที่ซับซ้อนยิ่งขึ้น โดยการเชื่อมโยงการใช้เหตุผลเข้ากับข้อมูล

ภายนอกแบบเรียลไทม์ [2] ทำให้ LLMs มีความหลากหลายและน่าเชื่อถือมากยิ่งขึ้น การเปลี่ยนแปลงนี้ถือเป็นการเปลี่ยนกระบวนทัศน์จากเดิมที่ LLMs ถูกมองว่าเป็นผู้ตอบสนองแบบพาสซีฟ (passive responders) ไปสู่การเป็นผู้ปฏิบัติการเชิงรุก (active agents) ที่สามารถวางแผนและโต้ตอบกับสภาพแวดล้อมได้ LLMs มาตรฐานส่วนใหญ่สร้างการตอบสนองตามคำสั่งที่ป้อนเข้ามาและข้อมูลที่ใช้ฝึกฝน [6] แต่ ReAct ได้นำเสนอขั้นตอน "การปฏิบัติ" (Action) ที่ช่วยให้ LLM สามารถเชื่อมต่อกับเครื่องมือภายนอกและ API ได้ [1] ความสามารถในการปฏิบัตินี้ได้เปลี่ยน LLM จากคลังความรู้แบบคงที่ให้กลายเป็นหน่วยงานแบบพลวัตที่สามารถค้นหาข้อมูลใหม่และดำเนินการเปลี่ยนแปลงได้ การเปลี่ยนแปลงกระบวนทัศน์นี้เปิดโอกาสให้เกิดการประยุกต์ใช้งานใหม่ๆ มากมาย ซึ่ง LLMs สามารถปฏิบัติงานได้จริง ไม่ใช่เพียงแค่ให้ข้อมูลเท่านั้น ซึ่งเป็นการขยายประโยชน์ใช้สอยในสถานการณ์จริงอย่างมีนัยสำคัญ อย่างไรก็ตาม การเปลี่ยนแปลงนี้ก็นำมาซึ่งความซับซ้อนใหม่ๆ ในด้านการควบคุม ความปลอดภัย และความสามารถในการคาดการณ์

ส่วนที่ 2: กลไกหลักของ ReAct

- **2.1. คำอธิบายวงจร ความคิด-การปฏิบัติ-การสังเกต (Thought-Action-Observation Cycle)**
  ReAct ดำเนินการบนวงจรการทำงานซ้ำๆ ของ ความคิด (Thought), การปฏิบัติ (Action), และ การสังเกต (Observation) [5] วงจรนี้เป็นหัวใจสำคัญของการทำงานของ ReAct:
  - **ความคิด (Thought):** LLM จะแสดงกระบวนการใช้เหตุผลแบบลูกโซ่แห่งความคิด (CoT) ออกมาเป็นคำพูด เพื่อทำความเข้าใจสถานการณ์ปัจจุบัน แบ่งย่อยงานออกเป็นส่วนเล็กๆ ที่จัดการได้ง่ายขึ้น วางแผน หรือปรับเปลี่ยนกลยุทธ์ตามผลการสังเกตการณ์ก่อนหน้า [1] นี่คือส่วนที่ "การใช้เหตุผล" ของ ReAct เข้ามามีบทบาท
  - **การปฏิบัติ (Action):** จากความคิดที่เกิดขึ้น LLM จะกำหนด

และระบุการดำเนินการเฉพาะงานที่จะต้องทำ [1] ซึ่งอาจเกี่ยวข้องกับการสืบค้นฐานข้อมูลความรู้ (เช่น Wikipedia API ดังที่กล่าวใน [1]) การใช้เครื่องคิดเลข การเรียกใช้ API ภายนอก หรือการโต้ตอบกับสภาพแวดล้อมซอฟต์แวร์
    - **การสังเกต (Observation):** LLM จะได้รับผลตอบรับจากสภาพแวดล้อมอันเป็นผลมาจากการปฏิบัติ [1] ผลการสังเกตนี้ (เช่น ผลการค้นหา, การตอบสนองจาก API, ข้อความแสดงข้อผิดพลาด) จะปรับปรุงความเข้าใจของ LLM และเป็นข้อมูลสำหรับขั้นตอน "ความคิด" (Thought) ถัดไป

  วงจรนี้สร้างวงจรป้อนกลับแบบพลวัต (dynamic feedback loop) ช่วยให้ตัวแทนสามารถประเมินความคืบหน้าของตนเอง เรียนรู้จากผลลัพธ์ของการปฏิบัติ และปรับเปลี่ยนแนวทางของตนเองจนกว่างานจะเสร็จสมบูรณ์หรือเป็นไปตามเงื่อนไขการสิ้นสุดที่กำหนดไว้ [5]
- **2.2. กลยุทธ์การออกแบบพรอมต์ (Prompt Engineering) สำหรับการนำ ReAct ไปใช้งาน**

  โดยทั่วไปแล้ว ReAct จะถูกนำไปใช้งานผ่านการออกแบบพรอมต์อย่างระมัดระวัง เพื่อชี้นำให้ LLM ปฏิบัติตามโครงสร้าง ความคิด-การปฏิบัติ-การสังเกต [5]
    - **การกระตุ้นแบบ Zero-Shot (Zero-Shot Prompting):** ReAct สามารถนำไปใช้ได้โดยใช้พรอมต์แบบ zero-shot ซึ่ง LLM จะได้รับคำแนะนำที่ชัดเจนเกี่ยวกับวิธีการปฏิบัติตามรูปแบบ ReAct โดยไม่มีตัวอย่างเฉพาะของงานนั้นๆ [13]
      - โครงสร้างตัวอย่างพรอมต์ (จาก [13]):
        - คำสั่งเริ่มต้นให้ใช้วิธีการแบบ ReAct
        - คำจำกัดความของรูปแบบ Thought:, Action:, Observation:
        - คำสั่งให้ดำเนินวงจรต่อไปจนกว่าปัญหาจะได้รับการแก้ไข
        - คำสั่งให้ระบุ Final Answer:
        - การย้ำให้แสดงผลลัพธ์ในรูปแบบที่กำหนดเสมอ

        - **เหตุผล:** สิ่งนี้ได้ผลเนื่องจาก LLMs ที่ปรับแต่งตามคำสั่ง (instruction-tuned LLMs) สามารถปฏิบัติตามคำแนะนำที่เป็นนามธรรมได้ [13]
    - **การกระตุ้นแบบ Few-Shot (Few-Shot Prompting):** แม้ว่า zero-shot จะเป็นไปได้ แต่การให้ตัวอย่างเล็กน้อย (ตัวอย่างในบริบทหนึ่งหรือสองตัวอย่างดังที่กล่าวถึงใน [3] สำหรับ ALFWorld และ WebShop) สามารถปรับปรุงประสิทธิภาพได้อย่างมีนัยสำคัญ โดยเฉพาะอย่างยิ่งสำหรับงานที่ซับซ้อนหรือเมื่อต้องการชี้นำ LLM เกี่ยวกับวิธีการใช้เครื่องมือเฉพาะ [15] แหล่งข้อมูล [16] ตั้งข้อสังเกตว่าหากไม่มีการปรับแต่งหรือตัวอย่างแบบ few-shot แบบจำลองมักจะสร้างเครื่องมือที่ไม่มีอยู่จริงขึ้นมา
    - เนื้อหาของพรอมต์ ([14]): พรอมต์ต้องกำหนดอย่างชัดเจน:
        - คำสั่ง/เป้าหมายหลัก
        - รูปแบบที่คาดหวังของ ความคิด, การปฏิบัติ, การสังเกต
        - คำแนะนำเกี่ยวกับสิ่งที่ต้องทำหากผลลัพธ์ไม่เป็นไปตามที่คาดหวังหรือการปฏิบัติล้มเหลว ซึ่งจะช่วยให้ LLM สร้างความคิดในลำดับถัดไปได้อย่างมีประสิทธิภาพมากขึ้น

ความสำเร็จของ ReAct ขึ้นอยู่กับความสามารถของ LLM ในการทำความเข้าใจและปฏิบัติตามรูปแบบพรอมต์ที่มีโครงสร้างและคำแนะนำที่เป็นนามธรรมที่กำหนดไว้อย่างมาก ReAct อาศัย LLM ในการสร้างส่วน "ความคิด" "การปฏิบัติ" และ "การสังเกต" ที่แตกต่างกันตามลำดับที่เฉพาะเจาะจง [5] หาก LLM ไม่สามารถปฏิบัติตามคำแนะนำเหล่านี้ได้ (เช่น ข้ามขั้นตอน สร้างการปฏิบัติโดยไม่มีความคิด หรือเข้าใจผิดเกี่ยวกับงานสำหรับแต่ละส่วน) กระบวนการทั้งหมดอาจล้มเหลว แหล่งข้อมูล [13] และ [13] เน้นว่าแบบจำลองที่ปรับแต่งตามคำสั่งเป็นกุญแจสำคัญสู่ความสำเร็จของ ReAct แบบ zero-shot ขณะที่ [16] ตั้งข้อสังเกตว่าปัญหาด้านประสิทธิภาพเกิดขึ้นและแบบจำลองอาจสร้างเครื่องมือที่ไม่มีอยู่จริงขึ้นมาหากไม่มีการกระตุ้นที่เหมาะสม (few-shot) หรือการปรับแต่ง สิ่งนี้บ่งบอกถึงการพึ่งพาคุณภาพและความสามารถของ LLM พื้นฐานอย่างมาก LLMs รุ่นใหม่ที่มีประสิทธิภาพสูงกว่าและมีความสามารถในการปฏิบัติตามคำสั่งที่ดี

กว่า มีแนวโน้มที่จะทำงานได้ดีขึ้นกับ ReAct นอกจากนี้ยังชี้ให้เห็นว่าการออกแบบพรอมต์สำหรับ ReAct ไม่ใช่เรื่องง่าย จำเป็นต้องมีการสร้างสรรค์อย่างรอบคอบและอาจต้องมีการปรับปรุงซ้ำๆ

- **2.3. วิธีที่ ReAct ผสานรวมเครื่องมือภายนอกและองค์ความรู้**
  ขั้นตอน "การปฏิบัติ" (Action) เป็นกลไกที่ตัวแทน ReAct ใช้ในการเชื่อมต่อกับเครื่องมือภายนอก, APIs, และฐานข้อมูลความรู้ [1] ตัวอย่างเครื่องมือ ได้แก่ Wikipedia API [1], เครื่องมือค้นหา, เครื่องคิดเลข, และฐานข้อมูล [15] การรวบรวมข้อมูลแบบพลวัตนี้ช่วยให้ LLM สามารถก้าวข้ามข้อมูลการฝึกอบรมแบบคงที่และเข้าถึงข้อมูลแบบเรียลไทม์และพลวัต ซึ่งมีความสำคัญอย่างยิ่งสำหรับงานที่ต้องการความรู้ที่เป็นปัจจุบันหรือการโต้ตอบกับสภาพแวดล้อมเฉพาะ [4] โดยการดึงข้อมูลภายนอก ReAct ช่วยในการอ้างอิงการตอบสนองของ LLM กับข้อมูลที่เป็นข้อเท็จจริง ซึ่งช่วยลดโอกาสในการเกิดการสร้างข้อมูลที่ไม่เป็นจริง (hallucinations) [1]
  ในขณะที่ "ความคิด" ให้การใช้เหตุผล และ "การปฏิบัติ" ให้ความสามารถในการดำเนินการ ขั้นตอน "การสังเกต" มีความสำคัญอย่างยิ่งในการเปิดใช้งานความสามารถในการปรับตัวและการแก้ไขข้อผิดพลาด ซึ่งเป็นคุณสมบัติที่มักขาดหายไปในวิธีการกระตุ้นที่ง่ายกว่า ใน CoT แบบง่าย ลูกโซ่แห่งความคิดจะดำเนินไปในลักษณะเชิงเส้นโดยไม่มีการป้อนกลับจากภายนอกจนกว่าจะมีการสร้างคำตอบสุดท้าย [4] หากเกิดข้อผิดพลาดในช่วงต้น ข้อผิดพลาดนั้นจะแพร่กระจายต่อไป ขั้นตอน "การสังเกต" ของ ReAct จะให้ผลลัพธ์ของการปฏิบัติแก่ LLM ไม่ว่าจะสำเร็จหรือไม่ก็ตาม [5] การป้อนกลับนี้ช่วยให้ LLM ในขั้นตอน "ความคิด" ถัดไปสามารถประเมินได้ว่าแผนกำลังดำเนินไปได้ด้วยดีหรือไม่ ข้อมูลที่ดึงมามีประโยชน์หรือไม่ หรือมีข้อผิดพลาดเกิดขึ้นที่ต้องได้รับการจัดการ [1] แหล่งข้อมูล [14] กล่าวถึงการระบุ "สิ่งที่ต้องทำหากผลลัพธ์ไม่สมเหตุสมผล" ในพรอมต์อย่างชัดเจนเพื่อช่วยในการสร้างความคิดในลำดับถัดไป กลไกการป้อนกลับแบบวนซ้ำนี้ทำให้ตัวแทน ReAct มีความยืดหยุ่นและสามารถจัดการกับงานที่ซับซ้อนและมีหลายขั้นตอนได้ดีขึ้น ซึ่งสภาพแวดล้อมมีความไม่

แน่นอนหรือแผนเริ่มต้นอาจไม่เหมาะสมที่สุด ช่วยให้เกิดการลองผิดลองถูกและการแก้ไขแนวทาง ซึ่งจำเป็นสำหรับการแก้ปัญหาที่แข็งแกร่ง

อย่างไรก็ตาม วงจร ความคิด-การปฏิบัติ-การสังเกต ที่มีโครงสร้าง แม้จะให้ความชัดเจนและความสามารถในการตีความ แต่อาจกำหนดข้อจำกัดที่จำกัดความยืดหยุ่นของ LLM ในบางสถานการณ์ ReAct บังคับใช้ลำดับการดำเนินการที่เฉพาะเจาะจง [5] แหล่งข้อมูล [4] ตั้งข้อสังเกตว่า "ข้อจำกัดเชิงโครงสร้างของ ReAct ลดความยืดหยุ่นในการกำหนดขั้นตอนการใช้เหตุผล" เมื่อเทียบกับ CoT ในบางบริบท แม้ว่าโครงสร้างนี้จะช่วยในการแก้ปัญหาอย่างเป็นระบบและลดการสร้างข้อมูลที่ไม่เป็นจริง แต่ก็อาจไม่เหมาะสมที่สุดสำหรับงานที่ต้องการเส้นทางการใช้เหตุผลที่ลื่นไหลและมีโครงสร้างที่เข้มงวดน้อยกว่า หรือในกรณีที่ "การปฏิบัติ" ไม่สามารถแบ่งแยกออกเป็นส่วนย่อยๆ ได้ง่าย สิ่งนี้ชี้ให้เห็นว่า ReAct ไม่ใช่ทางออกที่เหมาะสมสำหรับทุกสถานการณ์ สำหรับงานที่ต้องการการใช้เหตุผลที่สร้างสรรค์สูงหรือมีโครงสร้างน้อยกว่าโดยไม่มีการโต้ตอบกับสภาพแวดล้อมในทันที วิธีการอื่นๆ เช่น CoT แบบ thuần หรือแม้แต่ ToT (ซึ่งสำรวจหลายเส้นทาง) อาจเหมาะสมกว่าหรือเป็นส่วนเสริมซึ่งกันและกัน การเลือกกรอบการทำงานควรขึ้นอยู่กับลักษณะของงาน

ส่วนที่ 3: ข้อดีและจุดแข็งของกรอบการทำงาน ReAct

- **3.1. การปรับปรุงความแม่นยำและลดการสร้างข้อมูลที่ไม่เป็นจริง (Hallucinations)**

  ข้อได้เปรียบหลักประการหนึ่งของ ReAct คือความสามารถในการลดการสร้างข้อมูลที่ไม่เป็นจริงลงอย่างมาก และปรับปรุงความถูกต้องของข้อเท็จจริงในผลลัพธ์ของ LLM [1] กลไกนี้เกิดขึ้นได้โดยการอ้างอิงการใช้เหตุผลและการตอบสนองของ LLM กับข้อมูลที่ดึงมาจากภายนอกผ่านขั้นตอน "การปฏิบัติ" และ "การสังเกต" [1] แทนที่จะอาศัยเพียงความรู้เชิงพารามิเตอร์ (parametric knowledge) ของตนเอง (ซึ่งอาจล้าสมัยหรือไม่ถูกต้อง) LLM สามารถตรวจสอบหรือเสริม

ความรู้ของตนเองด้วยข้อมูลแบบเรียลไทม์ได้ หลักฐานจาก [1] ชี้ให้เห็นว่าในงานต่างๆ เช่น HotpotQA และ Fever นั้น ReAct สามารถเอาชนะปัญหาที่พบบ่อยเกี่ยวกับการสร้างข้อมูลที่ไม่เป็นจริงใน CoT ได้โดยการโต้ตอบกับ Wikipedia API นอกจากนี้ [17] ยังระบุว่า ReAct สามารถจัดการกับความไม่ถูกต้องของข้อเท็จจริงได้โดยให้แบบจำลองเข้าถึงความรู้ภายนอก

- **3.2. การเสริมสร้างการแก้ปัญหาและการตัดสินใจในงานที่ซับซ้อน**

  แนวทางที่มีโครงสร้างของวงจร ความคิด-การปฏิบัติ-การสังเกต ช่วยให้ LLMs สามารถแบ่งย่อยงานที่ซับซ้อนออกเป็นขั้นตอนย่อยๆ ที่จัดการได้ วางแผน ดำเนินการ และปรับเปลี่ยน [5] ReAct ช่วยให้เกิดการใช้เหตุผลแบบพลวัต ซึ่งแบบจำลองสามารถสร้าง รักษา และปรับเปลี่ยนแผนระดับสูงตามข้อมูลใหม่หรือผลลัพธ์ของขั้นตอนก่อนหน้าได้ [1] ซึ่งแตกต่างจากกระบวนการทำงานแบบคงที่ที่กำหนดไว้ล่วงหน้า ร่องรอยการใช้เหตุผลยังช่วยให้แบบจำลองสามารถจัดการกับข้อยกเว้นหรือปรับเปลี่ยนแผนตามสถานการณ์ได้ [1] หลักฐานจาก [3] รายงานประสิทธิภาพที่เหนือกว่าในการทดสอบเปรียบเทียบการตัดสินใจ เช่น ALFWorld และ WebShop เมื่อเทียบกับวิธีการเรียนรู้แบบเลียนแบบ (imitation learning) และการเรียนรู้แบบเสริมกำลัง (reinforcement learning) แม้จะมีตัวอย่างเพียงเล็กน้อยก็ตาม นอกจากนี้ [4] ยังตั้งข้อสังเกตถึงข้อได้เปรียบของ ReAct ในงานที่ต้องมีการสำรวจและการปฏิบัติที่มีประสิทธิภาพ

  จุดแข็งหลักของ ReAct ไม่ได้อยู่ที่ความสามารถในการใช้เหตุผลหรือการปฏิบัติเพียงอย่างเดียว แต่อยู่ที่การทำงานร่วมกันอย่างใกล้ชิดและสอดประสานกันของทั้งสองส่วนนี้ ส่วนหนึ่งช่วยเสริมอีกส่วนหนึ่งในวงจรที่ดี การใช้เหตุผล (ความคิด) ช่วยในการชักนำ ติดตาม และปรับปรุงแผนการปฏิบัติ ตลอดจนจัดการกับข้อยกเว้น [1] หากไม่มีการใช้เหตุผล การปฏิบัติอาจเป็นไปอย่างไร้ทิศทางหรือไม่สามารถปรับตัวได้ การปฏิบัติช่วยให้แบบจำลองสามารถเชื่อมต่อกับสภาพ

แวดล้อมภายนอก รวบรวมข้อมูลใหม่ และอ้างอิงการใช้เหตุผลของตนเอง [1] หากไม่มีการปฏิบัติ การใช้เหตุผลจะยังคงไม่ได้รับการอ้างอิงและมีแนวโน้มที่จะสร้างข้อมูลที่ไม่เป็นจริง แหล่งข้อมูล [1] ระบุว่า "ร่องรอยการใช้เหตุผลช่วยให้แบบจำลองชักนำ ติดตาม และปรับปรุงแผนการปฏิบัติ... ในขณะที่การปฏิบัติช่วยให้สามารถเชื่อมต่อและรวบรวมข้อมูลเพิ่มเติมได้" การทำงานร่วมกันนี้สร้างระบบที่แข็งแกร่งและน่าเชื่อถือมากกว่าระบบที่อาศัยเพียงการใช้เหตุผลภายใน (เช่น CoT) หรือการปฏิบัติแบบโต้ตอบโดยไม่มีการวางแผนอย่างลึกซึ้ง สิ่งนี้ชี้ให้เห็นว่าความก้าวหน้าในอนาคตของ AI แบบตัวแทนมีแนวโน้มที่จะยังคงสร้างขึ้นบนหลักการของการบูรณาการการใช้เหตุผลเข้ากับการโต้ตอบกับสภาพแวดล้อม

- **3.3. ความสามารถในการตีความและความน่าเชื่อถือ**
  การสร้างร่องรอยการใช้เหตุผล ("ความคิด") อย่างชัดเจนทำให้กระบวนการตัดสินใจของ LLM มีความโปร่งใสและมนุษย์สามารถตีความได้ง่ายขึ้น [1] มนุษย์สามารถตรวจสอบกระบวนการคิดเพื่อทำความเข้าใจว่า LLM บรรลุข้อสรุปได้อย่างไร หรือเหตุใดจึงดำเนินการอย่างใดอย่างหนึ่ง ซึ่งช่วยอำนวยความสะดวกในการแก้ไขข้อบกพร่องและการระบุแหล่งที่มาของข้อผิดพลาด [1] นอกจากนี้ยังทำให้ง่ายต่อการแยกแยะระหว่างข้อมูลที่ได้จากความรู้ภายในของแบบจำลองกับข้อมูลที่ได้รับจากสภาพแวดล้อมภายนอก [1] ซึ่งก่อให้เกิดความน่าเชื่อถือ
  แม้ว่าความสามารถในการตีความมักถูกอ้างถึงว่าเป็นเป้าหมายโดยตรง แต่ใน ReAct นั้น ความสามารถนี้เกิดขึ้นตามธรรมชาติจากกระบวนการแก้ปัญหาที่มีโครงสร้าง ซึ่งเป็นกุญแจสำคัญต่อประสิทธิภาพของมันเช่นกัน ความจำเป็นในการแบ่งย่อยปัญหาที่ซับซ้อนและการโต้ตอบกับสภาพแวดล้อมทำให้ต้องมีการวางแผนและการติดตามสถานะที่ชัดเจน (ความคิดและการสังเกต) การแสดงออกที่ชัดเจนนี้ ซึ่งออกแบบมาเพื่อให้ LLM มีประสิทธิภาพมากขึ้น ยังทำให้กระบวนการของมันเป็นที่เข้าใจได้สำหรับมนุษย์ [1]

"กระบวนการใช้เหตุผลที่แสดงออกมาเป็นคำพูด" [5] เป็นทั้งกลไกสำหรับ LLM และคำอธิบายสำหรับผู้ใช้ สิ่งนี้ชี้ให้เห็นว่าความพยายามที่จะทำให้ AI มีความสามารถมากขึ้น (โดยเฉพาะในงานที่ซับซ้อนและมีหลายขั้นตอน) สามารถดำเนินควบคู่ไปกับการทำให้ AI มีความโปร่งใสมากขึ้น ซึ่งเป็นการแก้ไขความท้าทายที่สำคัญในด้านจริยธรรมและการนำ AI ไปใช้ ในบริบทนี้ จึงไม่จำเป็นต้องมีการแลกเปลี่ยนระหว่างประสิทธิภาพและความสามารถในการตีความเสมอไป

- **3.4. ความหลากหลายในการใช้งานและความสามารถในการปรับตัวในแอปพลิเคชันต่างๆ**
  ReAct เป็นกระบวนทัศน์ทั่วไปที่สามารถนำไปประยุกต์ใช้กับชุดงานการใช้เหตุผลทางภาษาและการตัดสินใจที่หลากหลาย [1] ตัวแทน ReAct สามารถกำหนดค่าให้ทำงานร่วมกับเครื่องมือภายนอกและ APIs ที่หลากหลาย ซึ่งช่วยเพิ่มความสามารถรอบด้าน [5] ไม่จำเป็นต้องมีการกำหนดค่าแบบจำลองล่วงหน้าเพื่อเรียกใช้เครื่องมือ แม้ว่าการปรับแต่งจะสามารถปรับปรุงประสิทธิภาพได้ก็ตาม [5] ลักษณะพลวัตของการกำหนดเครื่องมือที่เหมาะสมและการปรับเปลี่ยนแผนตามการสังเกต ช่วยให้ตัวแทน ReAct สามารถปรับตัวเข้ากับความท้าทายใหม่ๆ และอุปสรรคที่ไม่คาดฝันได้ [5]
  แม้ว่า ReAct จะได้รับการออกแบบมาเพื่อความสามารถในการปรับตัว แต่ความสามารถในการปรับตัวอย่างมีประสิทธิภาพนั้นขึ้นอยู่กับข้อมูลที่เป็นประโยชน์จากผลตอบรับของสภาพแวดล้อมและความสามารถของ LLM ในการประมวลผลข้อมูลนั้น ความสามารถในการปรับตัวของ ReAct เกิดขึ้นจากความสามารถในการปรับเปลี่ยนแผนตามการสังเกต [5] แหล่งข้อมูล [4] ชี้ให้เห็นถึงการพึ่งพาที่สำคัญ: "ReAct ขึ้นอยู่กับข้อมูลที่ดึงมาเป็นอย่างมาก ผลการค้นหาที่ไม่ให้ข้อมูลจะทำให้การใช้เหตุผลของแบบจำลองคลาดเคลื่อนและนำไปสู่ความยากลำบากในการกู้คืนและปรับเปลี่ยนความคิดใหม่" หากเครื่องมือภายนอกส่งคืนข้อมูลที่ไม่ดี หรือหาก LLM ไม่สามารถตีความการ

สังเกตได้อย่างถูกต้องเพื่อปรับปรุงแผน ความสามารถในการปรับตัวก็จะล้มเหลว สิ่งนี้เน้นย้ำถึงจุดอ่อน ความแข็งแกร่งของตัวแทน ReAct ไม่ได้ถูกกำหนดโดย LLM เพียงอย่างเดียว แต่ยังขึ้นอยู่กับความน่าเชื่อถือและความสมบูรณ์ของเครื่องมือภายนอกและสภาพแวดล้อมที่มันโต้ตอบด้วย งานวิจัยในอนาคตอาจต้องมุ่งเน้นไปที่การทำให้ตัวแทนมีความยืดหยุ่นมากขึ้นต่อการสังเกตที่มีสัญญาณรบกวนหรือไม่ให้ข้อมูล หรือการพัฒนากลยุทธ์ที่ดีขึ้นสำหรับการสืบค้นและการเลือกเครื่องมือ

ส่วนที่ 4: การประยุกต์ใช้งานหลักและกรณีการใช้งานของ ReAct

- **4.1. งานที่ต้องใช้ความรู้เข้มข้น (Knowledge-Intensive Tasks)**
  - **การตอบคำถามที่ซับซ้อน (เช่น HotpotQA):** ReAct แสดงให้เห็นถึงประสิทธิภาพในงานตอบคำถามที่ต้องใช้การใช้เหตุผลหลายขั้นตอน (multi-hop reasoning) และการสังเคราะห์ข้อมูลจากแหล่งต่างๆ [1]
    - **กลไก:** ตัวแทนสามารถค้นหาข้อมูลส่วนต่างๆ ซ้ำๆ ใช้เหตุผลเกี่ยวกับความเชื่อมโยงของข้อมูลเหล่านั้น และสร้างคำตอบ คล้ายกับวิธีที่นักวิจัยที่เป็นมนุษย์อาจจัดการกับคำถามที่ซับซ้อน [1] แหล่งข้อมูล [17] ยังตั้งข้อสังเกตว่า ReAct เมื่อใช้ร่วมกับกลยุทธ์การถอดรหัส (decoding strategies) สามารถปรับปรุงการตอบคำถามแบบหลายขั้นตอนได้
  - **การตรวจสอบข้อเท็จจริง (เช่น Fever):** ReAct สามารถใช้เพื่อตรวจสอบความจริงของข้อกล่าวอ้างโดยการค้นหาและประเมินหลักฐานจากแหล่งความรู้ภายนอก เช่น Wikipedia อย่างกระตือรือร้น [1]
    - **กลไก:** ตัวแทนสามารถแบ่งย่อยข้อกล่าวอ้าง ค้นหาข้อเท็จจริงที่เกี่ยวข้อง จากนั้นจึงใช้เหตุผลว่าหลักฐานนั้นสนับสนุนหรือหักล้างข้อกล่าวอ้างดังกล่าว แหล่งข้อมูล [1] แสดงให้เห็นว่า ReAct มีประสิทธิภาพเหนือกว่า CoT ใน

งาน Fever

หัวข้อร่วมที่贯穿การประยุกต์ใช้งาน ReAct ที่ประสบความสำเร็จคือความจำเป็นในการแสวงหา รวบรวม และใช้เหตุผลเกี่ยวกับข้อมูลจากแหล่งภายนอกแบบพลวัตในหลายขั้นตอน งานต่างๆ เช่น HotpotQA (การตอบคำถามแบบหลายขั้นตอน), Fever (การตรวจสอบข้อเท็จจริง), ALFWorld (เกมเชิงโต้ตอบ) และ WebShop (การนำทางเว็บ) ล้วนเกี่ยวข้องมากกว่าการสืบค้นเพียงครั้งเดียวหรือลูกโซ่แห่งความคิดเชิงเส้น [1] งานเหล่านี้ต้องการให้ตัวแทนสำรวจ ค้นหาข้อมูลที่เกี่ยวข้อง ประเมินประโยชน์ของข้อมูลนั้น จากนั้นจึงตัดสินใจเกี่ยวกับขั้นตอนต่อไป วงจร ความคิด-การปฏิบัติ-การสังเกต เหมาะสมอย่างยิ่งสำหรับกระบวนการวนซ้ำนี้ สิ่งนี้บ่งชี้ว่า ReAct มีประสิทธิภาพโดยเฉพาะอย่างยิ่งสำหรับปัญหาที่ไม่สามารถแก้ไขได้ด้วยความรู้ที่มีอยู่ก่อนเพียงอย่างเดียว และในกรณีที่เส้นทางการแก้ปัญหาไม่ชัดเจนในทันที ซึ่งต้องมีการสำรวจและการโต้ตอบกับสภาพแวดล้อม ทำให้เป็นตัวเลือกที่แข็งแกร่งสำหรับงานวิจัยและการค้นพบที่เป็นอิสระมากขึ้น

- **4.2. การทดสอบเปรียบเทียบการตัดสินใจเชิงโต้ตอบ (Interactive Decision-Making Benchmarks)**
  - **เกมที่ใช้ข้อความเป็นฐาน (เช่น ALFWorld):** ReAct แสดงให้เห็นถึงประสิทธิภาพที่แข็งแกร่งในสภาพแวดล้อมที่ต้องการความเข้าใจคำสั่งที่เป็นภาษาธรรมชาติและการโต้ตอบกับโลกจำลองเพื่อบรรลุเป้าหมาย [1]
    - **กลไก:** ตัวแทนใช้ความคิดในการวางแผนลำดับการปฏิบัติ (เช่น "ไปที่ห้องครัว" "หยิบแอปเปิ้ล") และการสังเกตเพื่อทำความเข้าใจผลลัพธ์ของการปฏิบัติเหล่านี้ภายในสภาพแวดล้อมของเกม แหล่งข้อมูล [3] เน้นย้ำถึงการปรับปรุงอัตราความสำเร็จสัมบูรณ์ 34% ใน ALFWorld ด้วย ReAct
  - **การนำทางเว็บและการซื้อของ (เช่น WebShop):** ReAct สามารถนำทางเว็บไซต์ ค้นหาสินค้า และตัดสินใจตามเกณฑ์ของผู้ใช้ในสภาพแวดล้อมการซื้อของออนไลน์จำลอง [1]
    - **กลไก:** ตัวแทนสามารถใช้เหตุผลเกี่ยวกับคุณลักษณะของ

ผลิตภัณฑ์ เปรียบเทียบตัวเลือก และดำเนินการต่างๆ เช่น การคลิกลิงก์หรือการเพิ่มสินค้าลงในตะกร้าสินค้า แหล่งข้อมูล [3] แสดงให้เห็นถึงการปรับปรุงสัมบูรณ์ 10% ใน WebShop แหล่งข้อมูล [14] ให้ตัวอย่างโดยละเอียดว่า LLM ของ ReAct อาจค้นหาฐานข้อมูลผลิตภัณฑ์ ตรวจสอบผลิตภัณฑ์ และตรวจสอบเกณฑ์ได้อย่างไร

"การปฏิบัติ" ใน ReAct ไม่ได้จำกัดอยู่เพียงการกระทำทางกายภาพ แต่สามารถครอบคลุมการดำเนินการที่หลากหลาย รวมถึงการเรียก API การสืบค้นฐานข้อมูล การรันโค้ด หรือแม้แต่การเปลี่ยนแปลงสถานะภายใน ใน HotpotQA/Fever การปฏิบัติคือการสืบค้น Wikipedia [1] ใน ALFWorld การปฏิบัติคือคำสั่งภายในสภาพแวดล้อมของเกม [3] ใน WebShop การปฏิบัติคือการโต้ตอบกับองค์ประกอบของเว็บ [3] แหล่งข้อมูล [5] กล่าวถึงการใช้เครื่องมือที่สามารถมีได้ตั้งแต่ RAG ไปจนถึงเวิร์กโฟลว์แบบหลายตัวแทน และ [19] กล่าวถึง CodeAct ซึ่งตัวแทนสร้างและรันโค้ด ความยืดหยุ่นในการกำหนดสิ่งที่ถือเป็น "การปฏิบัติ" นี้ช่วยให้กรอบการทำงาน ReAct สามารถปรับให้เข้ากับโดเมนและประเภทปัญหาที่หลากหลายได้ ตรรกะหลักยังคงเหมือนเดิม แต่เครื่องมือเฉพาะและพื้นที่การปฏิบัติสามารถปรับแต่งได้ ซึ่งเน้นย้ำถึงลักษณะของมันในฐานะกระบวนทัศน์ทั่วไป [1]

- **4.3. ศักยภาพในโลกแห่งความเป็นจริงและตัวอย่าง**
  - **ตัวแทน AI สำหรับกระบวนการทำงานที่ซับซ้อน:** ReAct เป็นกรอบการทำงานสำหรับการสร้างตัวแทน AI ที่สามารถจัดการกับงานหลายขั้นตอนในโดเมนต่างๆ ตั้งแต่การสร้างเสริมการดึงข้อมูล (Retrieval Augmented Generation หรือ RAG) แบบง่ายๆ ไปจนถึงกระบวนการทำงานแบบหลายตัวแทนที่ซับซ้อน [5]
  - **การวิเคราะห์ข้อมูลและการรายงาน:** ตัวแทนสามารถใช้ ReAct เพื่อรวบรวมข้อมูลจากหลายแหล่ง (ฐานข้อมูล, APIs) ทำการคำนวณหรือวิเคราะห์ (โดยใช้เครื่องมือรันโค้ด) จากนั้นจึงสังเคราะห์รายงาน
  - **ผู้ช่วยส่วนตัว:** ผู้ช่วยส่วนตัวที่ซับซ้อนมากขึ้นซึ่งสามารถเข้าใจ

คำขอที่ซับซ้อน โต้ตอบกับบริการต่างๆ (เช่น การจอง, การจัดตารางเวลา) และให้คำอธิบายที่มีเหตุผลสำหรับการปฏิบัติของตน แหล่งข้อมูล [16] กล่าวถึงการผสานรวมกับแอปพลิเคชัน Office ใน Microsoft 365 Copilot เป็นตัวอย่างของการประยุกต์ใช้งานจริงของ LLMs ที่เชื่อมต่อกับเครื่องมือภายนอก

การปรับปรุงประสิทธิภาพที่รายงานของ ReAct (เช่น [3]) มักจะเปรียบเทียบกับวิธีการต่างๆ เช่น การกระตุ้นมาตรฐาน แบบจำลองที่เน้นการปฏิบัติเท่านั้น หรือแม้แต่ CoT ซึ่งโดยเนื้อแท้แล้วขาดองค์ประกอบหลักของ ReAct ไม่ว่าจะเป็นความสามารถในการโต้ตอบภายนอกหรือวงจรการใช้เหตุผล-การปฏิบัติที่สอดประสานกัน แหล่งข้อมูล [4] แสดงให้เห็นว่า ReAct โดยทั่วไปมีประสิทธิภาพดีกว่า "Act" (การปฏิบัติเท่านั้น) และสามารถแข่งขันได้หรือดีกว่า CoT ในบางงาน การเปรียบเทียบนี้เน้นย้ำว่าการเพิ่มองค์ประกอบที่ขาดหายไป (การใช้เหตุผลให้กับ Act หรือการโต้ตอบภายนอกให้กับ CoT) ผ่าน ReAct นำไปสู่ประสิทธิภาพที่ดีขึ้น ตัวอย่างเช่น CoT ประสบปัญหาการสร้างข้อมูลที่ไม่เป็นจริง [4] ซึ่ง ReAct บรรเทาลงได้ด้วยการอ้างอิงภายนอก แบบจำลองที่เน้นการปฏิบัติเท่านั้นไม่สามารถแบ่งย่อยเป้าหมายได้ [4] ซึ่ง ReAct แก้ไขด้วยความคิด สิ่งนี้เน้นย้ำถึงการมีส่วนร่วมพื้นฐานของการออกแบบที่ทำงานร่วมกันของ ReAct ไม่ใช่แค่การปรับปรุงทีละน้อย แต่เป็นการปรับปรุงโครงสร้างที่แก้ไขข้อจำกัดหลักของวิธีการก่อนหน้านี้ อย่างไรก็ตาม นั่นหมายความว่าเมื่อมีเกณฑ์เปรียบเทียบใหม่ๆ เกิดขึ้น (เช่น การเรียกใช้ฟังก์ชันที่ซับซ้อน) ข้อได้เปรียบสัมพัทธ์ของ ReAct จำเป็นต้องได้รับการประเมินใหม่อย่างต่อเนื่อง

ส่วนที่ 5: การวิเคราะห์เปรียบเทียบ: ReAct ในบริบทของเทคนิค LLM อื่นๆ

- **5.1. ReAct เปรียบเทียบกับ การกระตุ้นด้วยลูกโซ่แห่งความคิด (Chain-of-Thought - CoT) Prompting**
    - ***ภาพรวม CoT:*** การกระตุ้นแบบ CoT ส่งเสริมให้ LLMs สร้างร่องรอยการใช้เหตุผลทีละขั้นตอนก่อนที่จะได้คำตอบ ซึ่งช่วยปรับปรุงประสิทธิภาพในงานที่ต้องใช้การคำนวณทางคณิตศาสตร์ การใช้เหตุผลเชิงสามัญสำนึก และการใช้เหตุผลเชิงสัญลักษณ์ [4]

- **ความแตกต่างที่สำคัญและการทำงานร่วมกัน:**
  - **การโต้ตอบภายนอก:** CoT อาศัยความรู้ภายในของ LLM เป็นหลัก ในขณะที่ ReAct ผสานรวมการใช้เครื่องมือภายนอกและการรวบรวมข้อมูล [1] ทำให้ ReAct มีโอกาสเกิดการสร้างข้อมูลที่ไม่เป็นจริงน้อยกว่า [1]
  - **โครงสร้าง:** ReAct กำหนดวงจร ความคิด-การปฏิบัติ-การสังเกต แบบวนซ้ำอย่างเป็นทางการ ในขณะที่ CoT โดยทั่วไปเป็นกระบวนการคิดเชิงเส้นมากกว่า [21]
  - **การแพร่กระจายของข้อผิดพลาด:** CoT อาจประสบปัญหาการแพร่กระจายของข้อผิดพลาดหากขั้นตอนการใช้เหตุผลในช่วงต้นมีข้อบกพร่อง ขั้นตอนการสังเกตของ ReAct ช่วยให้สามารถแก้ไขและปรับเปลี่ยนได้ [1]
  - **การผสมผสาน:** ReAct ใช้ CoT โดยเนื้อแท้สำหรับขั้นตอน "ความคิด" [5] บทความต้นฉบับของ ReAct [1] แนะนำว่าการผสมผสานระหว่าง ReAct และ CoT (เพื่อให้สามารถใช้ทั้งความรู้ภายในและข้อมูลที่ได้รับจากภายนอก) สามารถเป็นแนวทางที่ดีที่สุดโดยรวมได้
- **ประสิทธิภาพสัมพัทธ์:** ReAct มีแนวโน้มที่จะมีประสิทธิภาพเหนือกว่า CoT ในงานที่ต้องใช้ความรู้เข้มข้นซึ่งต้องการการตรวจสอบจากภายนอก (เช่น Fever) แต่ CoT อาจเพียงพอหรือดีกว่าเล็กน้อยหากงานนั้นอาศัยการใช้เหตุผลภายในล้วนๆ และข้อมูลภายนอกสำหรับ ReAct มีสัญญาณรบกวนหรือไม่เป็นประโยชน์ (ใน [4] สำหรับ HotpotQA พบว่า CoT นำหน้าเล็กน้อยในบางตัวชี้วัด แม้ว่า ReAct จะลดการสร้างข้อมูลที่ไม่เป็นจริงได้ก็ตาม)

ReAct โดยการแสดงให้เห็นถึงพลังของการผสมผสานการใช้เหตุผลเข้ากับการปฏิบัติภายนอกผ่านการกระตุ้น มีแนวโน้มที่จะปูทางและเน้นย้ำถึงความจำเป็นในการแก้ปัญหาที่บูรณาการมากขึ้น เช่น การเรียกใช้ฟังก์ชันแบบเนทีฟ ReAct แสดงให้เห็นถึงประโยชน์ที่สำคัญของ LLMs ที่โต้ตอบกับเครื่องมือ [1] อย่างไรก็ตาม การบรรลุเป้าหมาย

นี้ผ่านการสร้างข้อความล้วนๆ (การแยกวิเคราะห์การปฏิบัติจากผลลัพธ์ของ LLM) อาจเปราะบางและไม่มีประสิทธิภาพ ความสำเร็จของ *แนวคิด* ReAct ย่อมนำนักวิจัยและนักพัฒนา LLMs ไปสู่การแสวงหาวิธีการที่แข็งแกร่ง มีประสิทธิภาพ และเป็นมาตรฐานมากขึ้น เพื่อให้ได้ผลลัพธ์เดียวกัน การเรียกใช้ฟังก์ชันแบบเนทีฟ ซึ่งให้ผลลัพธ์ที่มีโครงสร้างสำหรับการเรียกใช้เครื่องมือ เป็นคำตอบโดยตรงต่อความต้องการนี้ [6] ดังนั้น ReAct จึงสามารถมองได้ว่าเป็นกรอบการทำงานบุกเบิกซึ่งหลักการต่างๆ ได้ถูกดูดซับและปรับปรุงให้เป็นความสามารถของ LLM ที่เป็นเนทีฟมากขึ้น แม้ว่า "ReAct แบบ thuần" ผ่านการกระตุ้นอาจถูกใช้น้อยลงสำหรับการใช้เครื่องมือง่ายๆ ในปัจจุบัน แต่คุณูปการเชิงแนวคิดในการกำหนดรูปแบบ AI แบบตัวแทนนั้นมีความสำคัญอย่างยิ่ง

- **5.2. ReAct เปรียบเทียบกับ ต้นไม้แห่งความคิด (Tree of Thoughts - ToT)**
  - ***ภาพรวม ToT:*** ToT ขยายขอบเขตของ CoT โดยเปิดให้มีการสำรวจเส้นทางการใช้เหตุผลหลายเส้นทางพร้อมกัน คล้ายกับโครงสร้างต้นไม้ ช่วยให้ LLM สามารถสร้างความคิดระดับกลางหลายๆ ความคิด (กิ่งก้าน) ประเมินความคิดเหล่านั้น และใช้อัลกอริทึมการค้นหา (เช่น BFS หรือ DFS) หรือแม้แต่ตัวควบคุมที่ผ่านการฝึกฝนเพื่อตัดสินใจว่าจะติดตามเส้นทางใดหรือย้อนกลับจากเส้นทางใด [18]
  - **ความแตกต่างที่สำคัญ:**
    - **ความกว้างในการสำรวจ:** CoT และ ReAct โดยทั่วไปจะติดตามเส้นทางการใช้เหตุผลเพียงเส้นทางเดียวในแต่ละครั้ง (แม้ว่า ReAct จะสามารถวนซ้ำและแก้ไขได้) ToT จะสำรวจหลายเส้นทางพร้อมกันอย่างชัดเจน [21]
    - **การไตร่ตรองและการประเมินตนเอง:** ToT เกี่ยวข้องกับการประเมินตนเองอย่างรอบคอบมากขึ้นเกี่ยวกับเส้นทางความคิดต่างๆ ซึ่งมักจะต้องให้ LLM ให้คะแนนหรือประเมินความเป็นไปได้ของแต่ละกิ่งก้าน [22]
    - **ความซับซ้อนและกรณีการใช้งาน:** ToT ได้รับการออก

แบบมาสำหรับปัญหาที่ซับซ้อนซึ่งการสำรวจ การมองการณ์ไกลเชิงกลยุทธ์ หรือการแก้ปัญหาที่หลากหลายเป็นประโยชน์ [21] ReAct มุ่งเน้นไปที่การทำงานร่วมกันระหว่างการใช้เหตุผลและการปฏิบัติในลักษณะที่ต่อเนื่องและปรับเปลี่ยนได้

- ควรใช้อะไรเมื่อใด ([23]):
  - **ReAct:** เหมาะสำหรับงานที่ต้องการข้อมูลแบบเรียลไทม์ การใช้เครื่องมือ หรือการโต้ตอบกับสภาพแวดล้อม
  - **ToT:** เหมาะสมที่สุดเมื่อปัญหามีวิธีแก้ปัญหาที่เป็นไปได้หลายวิธี และต้องการให้แบบจำลองประเมินทางเลือกต่างๆ ก่อนที่จะตัดสินใจขั้นสุดท้าย
- **ศักยภาพในการผสมผสาน:** แหล่งข้อมูล [23] แนะนำว่า ToT และ ReAct สามารถนำมารวมกันได้ เช่น การใช้ ToT เพื่อสำรวจแผนการปฏิบัติที่เป็นไปได้ จากนั้นจึงดำเนินการและสังเกตการณ์ผ่านวงจรคล้าย ReAct

- **5.3. ReAct เปรียบเทียบกับ การเรียกใช้ฟังก์ชันแบบเนทีฟ (Native Function Calling)**
  - ***ภาพรวมการเรียกใช้ฟังก์ชัน:*** LLMs สมัยใหม่จำนวนมาก (OpenAI, Anthropic, Google, Mistral) ปัจจุบันรองรับความสามารถในการ "เรียกใช้ฟังก์ชัน" หรือ "การใช้เครื่องมือ" แบบเนทีฟ [16] ซึ่งช่วยให้นักพัฒนาสามารถกำหนดชุดของฟังก์ชัน/เครื่องมือที่มีอยู่ และ LLM สามารถสร้างอ็อบเจกต์ JSON ที่มีโครงสร้างซึ่งระบุว่าจะเรียกใช้ฟังก์ชันใดและด้วยอาร์กิวเมนต์ใด โดยอิงตามพรอมต์ของผู้ใช้ [6]
  - **ความแตกต่างที่สำคัญและวิวัฒนาการ:**
    - **การนำไปใช้งาน:** ReAct เป็นกรอบการทำงานการกระตุ้นเป็นหลักที่ชี้นำ LLM ให้สร้างข้อความที่แสดงถึงความคิดและการปฏิบัติ [15] การเรียกใช้ฟังก์ชันมักเป็นคุณลักษณะที่ผสานรวมกับแบบจำลองโดยตรงมากกว่า ซึ่งผลลัพธ์ของ

LLM สำหรับการใช้เครื่องมือเป็นอ็อบเจกต์ข้อมูลที่มีโครงสร้างแทนที่จะเป็นข้อความอิสระ [6]

    - **ความยืดหยุ่นเทียบกับโครงสร้าง:** ReAct สามารถมองได้ว่ามีความยืดหยุ่นมากกว่าเนื่องจาก LLM ใช้เหตุผลเป็นภาษาธรรมชาติเกี่ยวกับว่าจะดำเนินการใด [6] การเรียกใช้ฟังก์ชันอาศัยฟังก์ชันที่กำหนดไว้ล่วงหน้า ซึ่งสามารถมองได้ว่ามีโครงสร้างมากกว่า แต่อาจปรับตัวเข้ากับสถานการณ์ใหม่ๆ ที่ไม่ครอบคลุมโดยฟังก์ชันที่กำหนดไว้ได้น้อยกว่า [6]
    - **ความซับซ้อนและประสิทธิภาพ:** การเรียกใช้ฟังก์ชันมักถูกมองว่านำไปใช้งานได้ง่ายกว่าและอาจมีประสิทธิภาพมากกว่า (เร็วกว่า ใช้โทเค็นน้อยกว่า) สำหรับงานที่กำหนดไว้อย่างชัดเจน เนื่องจากข้ามขั้นตอนการสร้างภาษาธรรมชาติบางส่วนสำหรับการระบุการปฏิบัติ [6] แหล่งข้อมูล [16] ระบุอย่างชัดเจนว่าการเรียกใช้ฟังก์ชันเป็น "วิธีการแบบเนทีฟที่ใหม่กว่าและเหนือกว่า" และ ReAct "ส่วนใหญ่ถูกแทนที่ในช่วงปลายปี 2023 ด้วยเทคนิคการเรียกใช้ฟังก์ชันแบบเนทีฟ"
    - **ความชัดเจนในการใช้เหตุผล:** ขั้นตอน "ความคิด" ที่ชัดเจนของ ReAct ทำให้กระบวนการใช้เหตุผลโปร่งใส ในขณะที่ LLMs ที่ใช้การเรียกใช้ฟังก์ชันยังคงใช้เหตุผลภายใน แต่การใช้เหตุผลนี้อาจไม่แสดงออกมาอย่างชัดเจนเว้นแต่จะใช้ร่วมกับเทคนิคการกระตุ้นอื่นๆ
  - **ควรใช้อะไรเมื่อใด:**
    - **ReAct:** อาจยังคงมีประโยชน์สำหรับงานที่ต้องการการใช้เหตุผลที่ซับซ้อนและมีหลายขั้นตอน ซึ่งกระบวนการคิดที่ชัดเจนเป็นประโยชน์ในการชี้นำ LLM หรือเพื่อความสามารถในการตีความของมนุษย์ หรือเมื่อ "การปฏิบัติ" ที่แน่นอนไม่สามารถกำหนดไว้ล่วงหน้าเป็นฟังก์ชันที่มีโครงสร้างได้ง่าย
    - **การเรียกใช้ฟังก์ชัน:** โดยทั่วไปเป็นที่ต้องการสำหรับการ

ผสานรวม LLMs เข้ากับเครื่องมือภายนอกและ APIs เมื่องานค่อนข้างชัดเจนและสามารถจับคู่กับฟังก์ชันเฉพาะได้ มักจะมีความแข็งแกร่งและมีประสิทธิภาพมากกว่าสำหรับการผสานรวมดังกล่าว [6] แหล่งข้อมูล [16] แนะนำว่าการเรียกใช้ฟังก์ชันทำงานได้ดีกว่า ReAct ซึ่งทำงานได้เพียงประมาณ 30% ของเวลาและอาจช้า

ภูมิทัศน์ที่กำลังพัฒนานี้แสดงให้เห็นว่าไม่มีกรอบการทำงานใด "ดีที่สุด" อย่างถาวร การเปรียบเทียบแสดงให้เห็นถึงสาขาที่มีพลวัตซึ่งเทคนิคใหม่ๆ เกิดขึ้นและสร้างต่อยอดหรือเสนอทางเลือกให้กับเทคนิคก่อนหน้า แนวทาง "ที่ดีที่สุด" ขึ้นอยู่กับบริบทอย่างมากและอาจเปลี่ยนแปลงไปตามความก้าวหน้าของ LLM แหล่งข้อมูล [16] ตั้งข้อสังเกตว่า ReAct "ส่วนใหญ่ถูกแทนที่ในช่วงปลายปี 2023 ด้วยการเรียกใช้ฟังก์ชันแบบเนทีฟ" ผู้ปฏิบัติงานและนักวิจัยต้องประเมินเทคนิคที่มีอยู่อย่างต่อเนื่องเทียบกับข้อกำหนดของงานเฉพาะ ความสามารถของ LLM และข้อจำกัดในการนำไปใช้งาน สาขานี้ไม่หยุดนิ่งและแนวทางแบบผสมผสาน (เช่น [4] ที่แนะนำให้รวม ReAct และ CoT) มักจะเหมาะสมที่สุดนอกจากนี้ยังมีการแลกเปลี่ยนระหว่างการใช้เหตุผลที่ชัดเจนกับประสิทธิภาพ/ความเรียบง่าย ขั้นตอน "ความคิด" ที่ชัดเจนของ ReAct ให้ความสามารถในการตีความ แต่เพิ่มจำนวนโทเค็นและเวลาแฝง (latency) [16] การเรียกใช้ฟังก์ชัน แม้ว่าอาจมีความโปร่งใสน้อยกว่าในการตัดสินใจภายในเพื่อเรียกใช้ฟังก์ชัน แต่มักจะเร็วกว่าและง่ายกว่าสำหรับนักพัฒนา ReAct กำหนดให้ LLM สร้างข้อความสำหรับความคิด การปฏิบัติ และประมวลผลการสังเกต [16] การสร้างนี้ใช้เวลาและใช้โทเค็น ซึ่งส่งผลต่อต้นทุนและความเร็ว [6] การเรียกใช้ฟังก์ชันแบบเนทีฟจะส่งออกคำขอที่มีโครงสร้างโดยตรง ซึ่งช่วยลดภาระงานในการสร้างและแยกวิเคราะห์ภาษาธรรมชาติสำหรับการปฏิบัติ [6] แหล่งข้อมูล [6] ระบุอย่างชัดเจนว่า ReAct "ช้ากว่าเนื่องจากขั้นตอนการใช้เหตุผล" ในขณะที่การเรียกใช้ฟังก์ชัน "เร็วกว่าด้วยการรันฟังก์ชันโดยตรง" นี่คือทางเลือกในการออกแบบโดยธรรมชาติ: ให้ความสำคัญกับขั้นตอนการใช้เหตุผลที่มีรายละเอียดและตีความได้ (อาจมีต้นทุน/เวลาแฝงที่สูงขึ้น) หรือให้ความสำคัญกับการโต้ตอบกับเครื่องมือที่มีประสิทธิภาพและมีโครงสร้าง ทางเลือกที่

เหมาะสมที่สุดขึ้นอยู่กับว่าความซับซ้อนของงานนั้นคุ้มค่ากับภาระงานของการใช้เหตุผลที่ชัดเจนหรือไม่ หรือเป้าหมายหลักคือการเรียกใช้เครื่องมือที่รวดเร็วและเชื่อถือได้

- **ตารางที่ 1: การวิเคราะห์เปรียบเทียบกรอบการทำงานการใช้เหตุผล/การปฏิบัติของ LLM**

กรอบการทำงาน	**กลไกหลัก**	**ข้อได้เปรียบหลัก**	**ข้อจำกัดหลัก**	**กรณีการใช้งานในอุดมคติ**
Chain-of-Thought (CoT)	สร้างร่องรอยการใช้เหตุผลทีละขั้นตอนก่อนตอบ	ความเรียบง่าย, การใช้เหตุผลที่อธิบายได้	ไม่มีการเข้าถึงภายนอก, อาจเกิดการแพร่กระจายของข้อผิดพลาด	งานที่เน้นการใช้เหตุผลภายในล้วนๆ
ReAct	วงจร ความคิด-การปฏิบัติ-การสังเกต แบบวนซ้ำ โดยผสานรวมการใช้เหตุผลเข้ากับการปฏิบัติผ่านเครื่องมือภายนอก	การอ้างอิงภายนอก, ลดการสร้างข้อมูลเท็จ, การแก้ปัญหาที่ดีขึ้น, ตีความได้	ความซับซ้อน, ต้นทุนโทเค็น, ขึ้นอยู่กับคุณภาพพรอมต์/LLM, ปัญหาความน่าเชื่อถือ	งานเชิงโต้ตอบที่ต้องใช้ความรู้เข้มข้น, การตัดสินใจที่ต้องการการโต้ตอบกับสภาพแวดล้อม
Tree of Thoughts (ToT)	สำรวจเส้นทางการใช้เหตุผลหลายเส้นทางพร้อมกัน, ประเมินทางเลือกต่างๆ	การสำรวจ, การแก้ปัญหาที่ซับซ้อน, การค้นหาทางออกที่หลากหลาย	ความซับซ้อนในการคำนวณ, ต้องการฟังก์ชันการประเมินผล	การแก้ปัญหาที่ซับซ้อนซึ่งมีทางออกที่เป็นไปได้หลายทาง, งานที่ต้องการการ

				มองการณ์ไกลเชิงกลยุทธ์
Native Function Calling	LLM สร้างผลลัพธ์ที่มีโครงสร้าง (เช่น JSON) เพื่อเรียกใช้ฟังก์ชัน/เครื่องมือที่กำหนดไว้ล่วงหน้าโดยตรง	ประสิทธิภาพ, การผสานรวมเครื่องมือที่ง่ายขึ้น, ความแข็งแกร่งสำหรับงานที่กำหนดไว้ชัดเจน	ความยืดหยุ่นน้อยกว่า ReAct, การใช้เหตุผลอาจไม่ชัดเจนเท่า, จำกัดเฉพาะฟังก์ชันที่กำหนดไว้	การผสานรวมเครื่องมือและ API ที่คล่องตัวสำหรับงานเฉพาะทางที่มีโครงสร้างชัดเจน

ส่วนที่ 6: ข้อจำกัด ความท้าทาย และข้อควรพิจารณาที่สำคัญ

- **6.1. การใช้โทเค็นและต้นทุนการคำนวณ**
  ลักษณะการทำงานแบบวนซ้ำของ ReAct ซึ่งเกี่ยวข้องกับการสร้างความคิด การปฏิบัติ และการประมวลผลการสังเกตในหลายขั้นตอน มักจะต้องใช้โทเค็นและทรัพยากรการคำนวณมากกว่าเมื่อเทียบกับเทคนิคการกระตุ้นที่ง่ายกว่าหรือการเรียกใช้ฟังก์ชันโดยตรง [6] สิ่งนี้อาจนำไปสู่เวลาแฝงที่เพิ่มขึ้นและต้นทุนการดำเนินงานที่สูงขึ้น โดยเฉพาะอย่างยิ่งสำหรับงานที่ซับซ้อนซึ่งต้องมีการวนซ้ำหลายครั้ง แหล่งข้อมูล [6] ตั้งข้อสังเกตว่า ReAct "ใช้การคำนวณสูง"
- **6.2. การพึ่งพาคุณภาพของพรอมต์และความสามารถของ LLM**
  ประสิทธิภาพของ ReAct ขึ้นอยู่กับคุณภาพของพรอมต์เริ่มต้นและตัวอย่างที่ให้ไว้ (หากใช้ few-shot) อย่างมาก [16] พรอมต์ที่ออกแบบมาไม่ดีอาจนำไปสู่ประสิทธิภาพที่ไม่ดีเท่าที่ควรหรือไม่สามารถปฏิบัติตามโครงสร้าง ReAct ได้ ประสิทธิภาพของ ReAct ยังเชื่อมโยงกับความสามารถในการใช้เหตุผลและการปฏิบัติตามคำสั่งของ LLM พื้นฐาน [16] แหล่งข้อมูล [16] กล่าวถึงว่า ReAct "ถูกสร้างขึ้นโดยคำนึง

ถึง LLMs ซีรีส์ Davinci" และปัญหาด้านประสิทธิภาพจะทวีความรุนแรงขึ้นในแบบจำลองขนาดเล็ก ซึ่งบ่งบอกถึงการพึ่งพา LLMs ที่มีประสิทธิภาพสูงอย่างมาก

- **6.3. ปัญหาด้านประสิทธิภาพและบทบาทของการปรับแต่ง (Fine-tuning)**
  แหล่งข้อมูล [16] ระบุอย่างชัดเจนว่า "ปัญหาหลักของแบบจำลองตัวแทน ReAct คือมันทำงานได้เพียงประมาณ 30% ของเวลา ใช้เวลานานเกินไปในการทำงานง่ายๆ ให้สำเร็จ" ซึ่งชี้ให้เห็นถึงความกังวลอย่างมากเกี่ยวกับความน่าเชื่อถือในการใช้งานจริงหากไม่มีการตั้งค่าอย่างระมัดระวัง แม้ว่า ReAct จะสามารถนำไปใช้ได้ด้วยการกระตุ้น แต่การปรับแต่ง LLM บนวิถีการทำงานแบบ ReAct สามารถปรับปรุงประสิทธิภาพได้ [1] อย่างไรก็ตาม [16] ตั้งข้อสังเกตว่า "แอปพลิเคชันส่วนใหญ่ต้องการการปรับแต่ง ซึ่งนักพัฒนามักละเลย" และ "หากไม่มีการปรับแต่ง ประสิทธิภาพจะแย่กว่าพรอมต์ Chain-of-Thought (CoT)" ในบางกรณี
  ความเป็นจริงในการใช้งาน ReAct เป็นสมการที่ซับซ้อนซึ่งประกอบด้วยพลังของ LLM ทักษะในการออกแบบพรอมต์ และความเหมาะสมของงาน ข้อควรระวังด้านประสิทธิภาพที่สำคัญที่กล่าวถึง (เช่น [16] ที่ระบุว่า "ทำงานได้เพียงประมาณ 30% ของเวลา") ชี้ให้เห็นว่า ReAct ไม่ใช่โซลูชันแบบเสียบปลั๊กแล้วใช้งานได้ทันที ความสำเร็จของมันขึ้นอยู่กับความสมดุลที่ละเอียดอ่อน ReAct ต้องการการใช้เหตุผลและการปฏิบัติตามคำสั่งที่แข็งแกร่งจาก LLM [16] และต้องการทักษะในการออกแบบพรอมต์หรือการปรับแต่งอย่างละเอียด [1] นอกจากนี้ยังใช้โทเค็นมากกว่าและช้ากว่า [6] และความน่าเชื่อถือของเครื่องมือภายนอกก็มีความสำคัญอย่างยิ่ง [4] สิ่งนี้หมายความว่าแม้ ReAct จะมีแนวคิดที่ทรงพลัง แต่การนำไปใช้งานจริงก็เผชิญกับอุปสรรคที่สำคัญ อาจเหมาะสมที่สุดสำหรับสถานการณ์ที่ความซับซ้อนของงานนั้นคุ้มค่ากับภาระงานที่เพิ่มขึ้น และมีการลงทุนในแบบจำลองที่มีความสามารถสูงและการออกแบบพรอมต์/ตัวแทน

อย่างระมัดระวัง การเกิดขึ้นของทางเลือกที่ง่ายกว่าและแข็งแกร่งกว่า เช่น การเรียกใช้ฟังก์ชันแบบเนทีฟสำหรับการใช้เครื่องมือ [16] ยิ่งท้าทายตำแหน่งของ ReAct สำหรับบางแอปพลิเคชัน

- **6.4. ความท้าทายเกี่ยวกับข้อมูลและเครื่องมือภายนอก**
  การใช้เหตุผลของ ReAct อาจคลาดเคลื่อนได้หากเครื่องมือภายนอกส่งคืนผลการค้นหาที่ไม่ให้ข้อมูล ทำให้เข้าใจผิด หรือว่างเปล่า ทำให้แบบจำลองกู้คืนและปรับเปลี่ยนความคิดใหม่ได้ยาก [4] ประสิทธิภาพของกรอบการทำงานนี้ส่วนหนึ่งขึ้นอยู่กับความพร้อมใช้งานและการผสานรวมเครื่องมือภายนอกที่เกี่ยวข้องอย่างเหมาะสม [16] หากการปฏิบัติต้องการเครื่องมือที่ไม่มีอยู่ หรือหาก LLM สร้างเครื่องมือที่ไม่มีอยู่จริงขึ้นมา [16] กระบวนการจะล้มเหลว แหล่งข้อมูล [16] ยังตั้งข้อสังเกตถึงข้อจำกัด: "ทุกการปฏิบัติที่เป็นไปได้ที่แบบจำลองอาจต้องทำจะต้องระบุไว้ในพรอมต์อินพุต หากการปฏิบัติไม่รวมอยู่ในพรอมต์ แบบจำลองอาจไม่สามารถดำเนินการได้" ซึ่งชี้ให้เห็นถึงข้อจำกัดในความสามารถในการค้นพบการปฏิบัติใหม่ๆ ที่ไม่ได้บอกใบ้ไว้ในการตั้งค่าพรอมต์
- **6.5. ข้อควรพิจารณาด้านความเป็นส่วนตัวและความลำเอียงเมื่อโต้ตอบกับเครื่องมือภายนอก**
  การใช้เครื่องมือใน ReAct เพิ่มความเสี่ยงด้านความเป็นส่วนตัวโดยอาจมีการแบ่งปันข้อมูลผู้ใช้กับบริการภายนอกหรือ APIs ซึ่งอาจไม่ได้ปฏิบัติตามมาตรฐานความเป็นส่วนตัวที่สอดคล้องกัน [24] เครื่องมือภายนอกหรือฐานข้อมูลความรู้เองอาจมีความลำเอียง ซึ่งตัวแทน ReAct อาจนำไปรวมเข้ากับการใช้เหตุผลและผลลัพธ์ของตน [24] กระบวนการสะท้อนกลับ (reflection process) อาจยิ่งตอกย้ำความลำเอียงหากวงจรป้อนกลับมีความบิดเบือน การรับรองการปฏิบัติตามกฎระเบียบด้านการคุ้มครองข้อมูล (เช่น GDPR) จะซับซ้อนยิ่งขึ้นเมื่อข้อมูลไหลผ่านเครื่องมือภายนอก ซึ่งต้องมีการกำกับดูแลและการตรวจสอบอย่างรอบคอบ [24]
  แม้ว่า ReAct จะมุ่งหวังความสามารถในการตีความผ่านร่องรอยความ

คิด แต่เครื่องมือภายนอกที่มันโต้ตอบด้วยอาจเป็น "กล่องดำ" (black boxes) ในตัวเอง ซึ่งนำมาซึ่งความลำเอียงที่ไม่สามารถอธิบายได้ หรือการรั่วไหลของความเป็นส่วนตัว ReAct อาศัยเครื่องมือภายนอกสำหรับข้อมูลและการปฏิบัติ [1] การทำงานภายใน แหล่งข้อมูล และความลำเอียงของเครื่องมือของบุคคลที่สามเหล่านี้อาจไม่โปร่งใส [24] ดังนั้น แม้ว่าการใช้เหตุผลของ LLM ใน ReAct จะโปร่งใส แต่ข้อมูลหรือผลลัพธ์การปฏิบัติที่ได้รับจากเครื่องมืออาจมีความลำเอียงหรือละเมิดความเป็นส่วนตัวโดยที่ LLM หรือผู้ใช้ไม่ทราบถึงแหล่งที่มาของปัญหาอย่างเต็มที่ แหล่งข้อมูล [24] กล่าวถึงโดยเฉพาะว่าการใช้เครื่องมือสามารถสืบทอดความลำเอียงจากเครื่องมือเฉพาะ และการปฏิบัติตามมาตรฐานความเป็นส่วนตัวของบุคคลที่สามเป็นข้อกังวล การรับรองความน่าเชื่อถือและความปลอดภัยโดยรวมของตัวแทน ReAct จึงไม่เพียงแต่ต้องมีการตรวจสอบและติดตามองค์ประกอบ LLM เท่านั้น แต่ยังรวมถึงระบบนิเวศทั้งหมดของเครื่องมือภายนอกที่มันพึ่งพาด้วย ซึ่งเพิ่มความซับซ้อนให้กับการสร้างระบบที่ใช้ ReAct อย่างมีความรับผิดชอบ

นอกจากนี้ยังมีความเสี่ยงจากการล้าสมัยเนื่องจากวิวัฒนาการอย่างรวดเร็วของ LLM ReAct ได้รับการออกแบบโดยคำนึงถึงความสามารถ (และข้อจำกัด) บางประการของ LLM (เช่น ซีรีส์ Davinci ตาม [16]) ในขณะที่ LLMs พัฒนาอย่างรวดเร็ว โดยมีความสามารถแบบเนทีฟ เช่น การเรียกใช้ฟังก์ชันกลายเป็นมาตรฐาน กลไกการกระตุ้นเฉพาะของ ReAct อาจมีความเกี่ยวข้องหรือเหมาะสมน้อยลง ReAct เป็นวิธีการใหม่ในการกระตุ้นการใช้เหตุผลและการปฏิบัติเมื่อ LLMs ขาดกลไกที่ตรงกว่าสำหรับการใช้เครื่องมือ [1] แบบจำลองรุ่นใหม่มีวิธีการในตัว ซึ่งมักจะมีประสิทธิภาพมากกว่า ในการบรรลุการโต้ตอบกับเครื่องมือ [16] แหล่งข้อมูล [16] ระบุว่า ReAct "ส่วนใหญ่ถูกแทนที่ในช่วงปลายปี 2023 ด้วยเทคนิคการเรียกใช้ฟังก์ชันแบบเนทีฟ" สิ่งนี้เน้นย้ำถึงประเด็นร่วมในสาขาเทคโนโลยีที่พัฒนาอย่างรวดเร็ว: เทคนิคบุกเบิกสามารถกลายเป็นแนวคิดพื้นฐานได้อย่าง

รวดเร็ว ซึ่งการนำไปใช้งานโดยตรงจะถูกแทนที่ด้วยวิธีการที่บูรณาการหรือมีประสิทธิภาพมากกว่า *หลักการ* ของ ReAct (การผสานการใช้เหตุผลและการปฏิบัติ) ยังคงมีความสำคัญ แต่ *กรอบการทำงานการกระตุ้น* เฉพาะอาจเห็นการใช้งานโดยตรงลดลงเพื่อสนับสนุนแนวทางสถาปัตยกรรมที่ใหม่กว่า

ส่วนที่ 7: ภูมิทัศน์ที่กำลังพัฒนา: อนาคตของ ReAct และ AI แบบตัวแทน (Agentic AI)

- **7.1. แนวโน้มการวิจัยในปัจจุบันและศักยภาพในการปรับปรุง** การวิจัยในปัจจุบันมุ่งเน้นไปที่การผสานรวมวงจรการทำงานซ้ำๆ ของ ReAct เข้ากับกลไกการวางแผนและการใช้เหตุผลที่ซับซ้อนยิ่งขึ้น ซึ่งอาจดึงแนวคิดจาก ToT มาใช้ในการสำรวจทางเลือกในการปฏิบัติ หรือใช้การสะท้อนตนเอง (self-reflection) เพื่อปรับปรุงแผน [18] ("Self-Reflection introduces cognitive loops") นอกจากนี้ยังมีการวิจัยเกี่ยวกับการเลือกและการใช้เครื่องมือที่ชาญฉลาดและพลวัตมากขึ้น ซึ่งอาจเกี่ยวข้องกับการให้ตัวแทนเรียนรู้ว่าเครื่องมือใดดีที่สุดสำหรับงานย่อยใด หรือวิธีการรวมเครื่องมือต่างๆ อย่างมีประสิทธิภาพมากขึ้น [19] ("Modern Tool Use structures to choose and utilize specialized tools") แม้ว่า [12] จะอ้างถึง "REACT" ที่แตกต่างกันสำหรับการแก้ไของค์ความรู้ แต่ความท้าทายของการปรับตัวเกินควร (overfitting) และการรับรองการอัปเดตที่ตรงเป้าหมายนั้นเกี่ยวข้องกับการเรียนรู้ของตัวแทน ตัวแทนคล้าย ReAct ในอนาคตอาจรวมกลไกเพื่อให้แน่ใจว่าการเรียนรู้จากการสังเกตนั้นเป็นแบบเฉพาะที่และไม่ส่งผลกระทบในทางลบต่อความสามารถทั่วไป สถาปัตยกรรมแบบผสมผสาน (hybrid architectures) ที่กำลังเกิดขึ้น ซึ่งเชื่อมโยงส่วนต่อประสานภาษาธรรมชาติเข้ากับท่อส่งการดำเนินการอัตโนมัติ (autonomous execution pipelines) [25] ชี้ให้เห็นว่าหลักการของ ReAct อาจเป็นส่วนหนึ่งของระบบที่ซับซ้อนมากขึ้น
- **7.2. บทบาทของ ReAct ในระบบหลายตัวแทน (Multi-Agent Systems)**

หลักการของ ReAct (การใช้เหตุผลเกี่ยวกับเป้าหมาย, การปฏิบัติ, การสังเกตผลลัพธ์) สามารถขยายไปสู่ระบบหลายตัวแทน ซึ่งตัวแทนจำเป็นต้องประสานงาน สื่อสาร และทำงานร่วมกันหรือแข่งขันกันเพื่อแก้ปัญหาที่ซับซ้อน [19] ("Multi-Agent workflows orchestrate collaborative or competitive interactions") ในการตั้งค่าแบบหลายตัวแทน ตัวแทนที่แตกต่างกันอาจมีความสามารถคล้าย ReAct ที่ปรับให้เหมาะกับบทบาทเฉพาะ โดยมีตัวแทนประสานงานกลางหรือโปรโตคอล [19] ("specialized subagents that collaborate") หลักการหลักของ ReAct (การใช้เหตุผลและการปฏิบัติที่สอดประสานกัน) กำลังกลายเป็นรากฐานสำหรับ AI แบบตัวแทนในวงกว้าง แม้ว่าเทคนิคการกระตุ้น ReAct เฉพาะอาจมีการพัฒนาหรือถูกแทนที่ แต่แนวคิดพื้นฐานของ LLM ที่คิด ปฏิบัติ และสังเกต ยังคงเป็นรากฐานที่สำคัญสำหรับแนวคิด AI แบบตัวแทนขั้นสูงจำนวนมาก แนวคิดต่างๆ เช่น การสะท้อนตนเอง (การวิพากษ์วิจารณ์การปฏิบัติในอดีตเพื่อปรับปรุงการปฏิบัติในอนาคต - [18]), Agentic RAG (การค้นหาและการสังเคราะห์เชิงรุก - [19]), CodeAct (การสร้างและรันโค้ด - [19]), และเวิร์กโฟลว์แบบหลายตัวแทน (Multi-Agent Workflows - [19]) ล้วนอาศัยความสามารถของตัวแทนในการใช้เหตุผลเกี่ยวกับงาน ดำเนินการบางรูปแบบ (แม้ว่าจะเป็นภายใน เช่น การวิพากษ์วิจารณ์ตนเอง) และประมวลผลผลลัพธ์ ไม่ว่าโดยนัยหรือโดยชัดแจ้ง ReAct ได้ให้แบบจำลองที่ชัดเจนในช่วงแรกสำหรับวงจรนี้ คุณูปการเชิงแนวคิดของ ReAct จึงยังคงอยู่ ระบบตัวแทนในอนาคตมีแนวโน้มที่จะรวมวงจรหลักของมันเข้าไว้ด้วยกันในรูปแบบที่ซับซ้อนและบูรณาการมากขึ้น โดยก้าวข้ามการกระตุ้นแบบง่ายๆ ไปสู่คุณลักษณะทางสถาปัตยกรรมที่ฝังลึกยิ่งขึ้น

- **7.3. AI แบบตัวแทนและเส้นทางสู่ระบบอัตโนมัติมากขึ้น**
  ReAct เป็นก้าวหนึ่งไปสู่ตัวแทน AI ที่เป็นอิสระมากขึ้น ซึ่งสามารถตัดสินใจ โต้ตอบกับเครื่องมือ และดำเนินการโดยต้องการการป้อนข้อมูลจากมนุษย์น้อยลงอย่างต่อเนื่อง [18] ("agentic coding

minimizes the need for continuous human oversight") แนวคิดของ Agentic RAG [19] ชี้ให้เห็นถึงตัวแทนที่ไม่เพียงแค่ดึงข้อมูลแบบพาสซีฟ แต่ยังค้นหา ประเมิน และสังเคราะห์ข้อมูลอย่างกระตือรือร้น ซึ่งเป็นกระบวนการที่สอดคล้องกับวงจร ความคิด-การปฏิบัติ-การสังเกต ของ ReAct เป็นอย่างดี การสร้างต่อยอดจาก ReAct ทำให้ CodeAct ช่วยให้ตัวแทนสามารถสร้างและรันโค้ด ซึ่งเป็นการขยายขีดความสามารถในการแก้ปัญหาในโดเมนทางเทคนิค [19] แผนงานในอนาคต [25] เกี่ยวข้องกับการพัฒนาโครงสร้างพื้นฐานสำหรับระบบ AI แบบตัวแทนที่น่าเชื่อถือ อธิบายได้ และทำงานร่วมกันได้ ซึ่งหลักการของ ReAct ในด้านการใช้เหตุผลและการปฏิบัติที่ตีความได้จะมีบทบาทสำคัญ

การผลักดันไปสู่ความเป็นอิสระมากขึ้นทำให้ความน่าเชื่อถือและการควบคุมมีความสำคัญยิ่งขึ้น ในขณะที่ตัวแทนคล้าย ReAct มีความเป็นอิสระและมีความสามารถในการปฏิบัติที่ซับซ้อนมากขึ้น (เช่น CodeAct, ระบบหลายตัวแทน) ความจำเป็นในการจัดการข้อผิดพลาดที่แข็งแกร่ง กลไกความปลอดภัย และการกำกับดูแลของมนุษย์ก็ยิ่งมีความสำคัญมากขึ้น ReAct มีข้อจำกัดด้านความน่าเชื่อถืออยู่แล้ว [4] ความเป็นอิสระที่มากขึ้น [25] หมายความว่าตัวแทนสามารถทำงานโดยมีการกำกับดูแลโดยตรงจากมนุษย์น้อยลง หากตัวแทนอิสระที่ใช้หลักการ ReAct ทำผิดพลาดในการประมวลผลความคิด การปฏิบัติ หรือการสังเกต ผลที่ตามมาอาจมีความสำคัญมากกว่าในระบบที่มีความเป็นอิสระน้อยกว่า แหล่งข้อมูล [26] เน้นย้ำถึง "ความปลอดภัย การปรับแนว และการลดความลำเอียง" และ "การจัดการความปลอดภัยและความเสี่ยง" ว่าเป็นแนวโน้มสำคัญ ขณะที่ [25] กล่าวถึงความจำเป็นในการ "ติดตาม แก้ไขข้อบกพร่อง และปรับแนวพฤติกรรมของตัวแทนให้สอดคล้องกับเจตนาของมนุษย์" การวิจัยในอนาคตจึงต้องมุ่งเน้นอย่างมากในการทำให้กรอบการทำงานแบบตัวแทนเหล่านี้ไม่เพียงแต่มีความสามารถมากขึ้นเท่านั้น แต่ยังต้องมีความน่าเชื่อถือ คาดการณ์ได้ และควบคุมได้มากขึ้นอย่างมีนัยสำคัญ

ความสามารถในการตีความที่นำเสนอโดยร่องรอยความคิดของ ReAct จะมีคุณค่าในส่วนนี้ แต่ก็ไม่เพียงพอในตัวเอง

- **7.4. การจัดการกับข้อจำกัด: ประสิทธิภาพและความน่าเชื่อถือ**
  แนวโน้มไปสู่ LLMs ที่มีขนาดเล็กและมีประสิทธิภาพมากขึ้น [26] อาจทำให้กรอบการทำงานคล้าย ReAct เข้าถึงได้ง่ายขึ้นและคุ้มค่ามากขึ้น หากแบบจำลองเหล่านี้สามารถรักษาความสามารถในการใช้เหตุผลที่แข็งแกร่งไว้ได้ ดังที่เห็นใน [17] การรวม ReAct เข้ากับกลยุทธ์การถอดรหัสขั้นสูงสามารถเพิ่มความเที่ยงตรงและความแม่นยำ ซึ่งชี้ให้เห็นถึงแนวทางในการปรับปรุงความน่าเชื่อถือ การมุ่งเน้นอย่างต่อเนื่องเกี่ยวกับกับการกำกับดูแลที่แข็งแกร่ง ความโปร่งใส และแนวทางปฏิบัติ AI ที่มีความรับผิดชอบ [26] จะมีความสำคัญอย่างยิ่งสำหรับการพัฒนาตัวแทนที่ได้รับแรงบันดาลใจจาก ReAct ที่ก้าวหน้าและเป็นอิสระมากขึ้น
  ความเชี่ยวชาญเฉพาะทางและการแบ่งส่วนงาน (modularity) เป็นแนวทางหนึ่งในอนาคต แนวคิดเรื่อง "ตัวแทนย่อยที่เชี่ยวชาญเฉพาะทาง" (specialized subagents) [25] หรือตัวแทนที่มี "บทบาทเฉพาะทาง" (specialized roles) [19] ชี้ให้เห็นว่าตัวแทน ReAct แบบเดี่ยวที่พยายามทำทุกอย่างอาจมีประสิทธิภาพน้อยกว่าระบบของตัวแทนที่ทำงานร่วมกันและเชี่ยวชาญเฉพาะทาง ซึ่งแต่ละตัวอาจใช้หลักการคล้าย ReAct สำหรับโดเมนเฉพาะของตน LLM เดียวที่พยายามเชี่ยวชาญเครื่องมือและโดเมนการใช้เหตุผลทั้งหมดสำหรับงานที่ซับซ้อนต้องเผชิญกับความท้าทายอย่างใหญ่หลวง การแบ่งย่อยปัญหาที่ซับซ้อนออกเป็นปัญหาย่อยที่สามารถจัดการได้โดยตัวแทนที่เชี่ยวชาญเฉพาะทางเป็นแนวทางทางวิศวกรรมทั่วไป ตัวแทนที่เชี่ยวชาญเฉพาะทางแต่ละตัวสามารถใช้วงจรคล้าย ReAct ที่ปรับให้เหมาะกับเครื่องมือและโดเมนความรู้เฉพาะของตนได้ แหล่งข้อมูล [19] กล่าวถึง "เวิร์กโฟลว์แบบหลายตัวแทน" ที่ส่งเสริมการแบ่งส่วนงานและความสามารถในการปรับขนาดอย่างชัดเจน อนาคตของ ReAct จึงอาจไม่ได้อยู่ที่ตัวแทน ReAct เดียวที่ทรงพลังทุกด้าน แต่เป็น

หลักการของมันที่ถูกนำไปใช้ภายในองค์ประกอบของระบบ AI แบบหลายตัวแทนที่มีขนาดใหญ่ขึ้นและแบ่งส่วนงานได้ ซึ่งอาจช่วยปรับปรุงความแข็งแกร่งและความสามารถโดยรวมของระบบ

ส่วนที่ 8: สรุป: การสังเคราะห์ผลกระทบและศักยภาพของ ReAct

- **8.1. สรุปผลการค้นพบที่สำคัญ**
  ReAct ถูกนิยามขึ้นจากหลักการของการผสานการทำงานระหว่างการใช้เหตุผล (ความคิด) และการปฏิบัติที่อ้างอิงกับสภาพแวดล้อม (การปฏิบัติ-การสังเกต) ข้อได้เปรียบหลักของมันคือการลดการสร้างข้อมูลที่ไม่เป็นจริง, การปรับปรุงการแก้ปัญหาในงานที่ซับซ้อน, การเพิ่มความสามารถในการตีความ และความหลากหลายในการใช้งาน การประยุกต์ใช้งานที่สำคัญ ได้แก่ การตอบคำถามที่ต้องใช้ความรู้เข้มข้น, การตรวจสอบข้อเท็จจริง และการตัดสินใจเชิงโต้ตอบ อย่างไรก็ตาม ReAct ก็มีข้อจำกัดที่สำคัญ เช่น ต้นทุนด้านโทเค็นและการคำนวณ, การพึ่งพาคุณภาพของพรอมต์และ LLM, ปัญหาด้านความน่าเชื่อถือ และการเกิดขึ้นของการเรียกใช้ฟังก์ชันแบบเนทีฟซึ่งเป็นทางเลือกที่มีประสิทธิภาพมากกว่าสำหรับการใช้งานเครื่องมือในหลายกรณี
- **8.2. คุณูปการที่ยั่งยืนของ ReAct ต่อความก้าวหน้าของ LLM**
  ReAct มีบทบาทสำคัญในฐานะกรอบการทำงานบุกเบิกที่แสดงให้เห็นว่า LLMs สามารถเปลี่ยนจากผู้สร้างข้อความแบบพาสซีฟไปเป็นตัวแทนเชิงรุกที่สามารถวางแผนและโต้ตอบกับโลกได้ แม้ว่าเทคนิคการกระตุ้น ReAct เฉพาะอาจถูกแทนที่ในบางแอปพลิเคชันด้วยความสามารถของแบบจำลองที่บูรณาการมากขึ้น (เช่น การเรียกใช้ฟังก์ชันแบบเนทีฟ) แต่ *ความก้าวหน้าเชิงแนวคิด* ของการสอดแทรกการใช้เหตุผลที่ชัดเจนเข้ากับการปฏิบัติและการสังเกตยังคงมีอิทธิพลอย่างสูงต่อการพัฒนา AI แบบตัวแทนอย่างต่อเนื่อง หลักการที่อยู่เบื้องหลัง ReAct ยังคงเป็นข้อมูลสำหรับการวิจัยเกี่ยวกับตัวแทนที่ซับซ้อนมากขึ้น รวมถึงตัวแทนที่มีความสามารถในการสะท้อนตนเอง การวางแผนขั้นสูง และการทำงานร่วมกันแบบหลายตัวแทน
- **8.3. ข้อคิดสุดท้ายเกี่ยวกับทิศทางของ AI แบบตัวแทน**

วิวัฒนาการอย่างรวดเร็วของสาขานี้แสดงให้เห็นว่ากรอบการทำงานเช่น ReAct ทำหน้าที่เป็นก้าวสำคัญ แม้ว่าความท้าทายในด้านความน่าเชื่อถือ ประสิทธิภาพ และความปลอดภัยจะยังคงมีอยู่ แต่เส้นทางสู่ตัวแทน AI ที่มีความสามารถ เป็นอิสระ และน่าเชื่อถือมากขึ้น โดยสร้างขึ้นจากแนวคิดพื้นฐานเช่นเดียวกับที่ปรากฏใน ReAct ยังคงก้าวหน้าต่อไป จุดเน้นน่าจะอยู่ที่การผสานรวมวงจรการใช้เหตุผล-การปฏิบัติเหล่านี้เข้ากับสถาปัตยกรรม AI อย่างลึกซึ้งยิ่งขึ้น และรับรองว่าพวกมันทำงานได้อย่างแข็งแกร่งและมีความรับผิดชอบ

- **ตารางที่ 2: สรุปกรอบการทำงาน ReAct: คุณลักษณะหลัก ข้อได้เปรียบ และข้อจำกัด**

แง่มุม	**คำอธิบาย**
กลไกหลัก	วงจร ความคิด-การปฏิบัติ-การสังเกต (Thought-Action-Observation cycle)
เป้าหมายหลัก	ผสานการใช้เหตุผลและการปฏิบัติเข้าด้วยกัน
หมวดหมู่ข้อได้เปรียบหลัก	ลดการสร้างข้อมูลเท็จ, การแก้ปัญหาที่ดีขึ้น, ความสามารถในการตีความ, ความหลากหลายในการใช้งาน
หมวดหมู่ข้อจำกัดหลัก	ต้นทุนโทเค็น/การคำนวณ, การพึ่งพาคุณภาพพรอมต์/LLM, ปัญหาความน่าเชื่อถือ, การแข่งขันจากการเรียกใช้ฟังก์ชันแบบเนทีฟ
สถานะปัจจุบัน	มีอิทธิพลเชิงแนวคิด, เทคนิคการกระตุ้นเฉพาะส่วนหนึ่งถูกแทนที่ด้วยการเรียกใช้ฟังก์ชันแบบเนทีฟสำหรับการใช้เครื่องมือ แต่หลักการยังคงเป็นพื้นฐานสำหรับ AI

	แบบตัวแทนขั้นสูง

ผลงานที่อ้างอิง

1. REACT: SYNERGIZING REASONING AND ACTING ... - OpenReview, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://openreview.net/pdf?id=WE_vluYUL-X
2. ReAct: Synergizing Reasoning and Acting in Language Models - ExplainPrompt, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.explainprompt.com/papers/react
3. ReAct: Synergizing Reasoning and Acting in Language Models - arXiv, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://arxiv.org/pdf/2210.03629
4. ReAct - Prompt Engineering Guide, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.promptingguide.ai/techniques/react
5. What is a ReAct Agent? | IBM, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.ibm.com/think/topics/react-agent
6. React Agents vs Function Calling Agents - PureLogics, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://purelogics.com/react-agents-vs-function-calling-agents/
7. Next.js by Vercel - The React Framework, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://nextjs.org/
8. React, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://react.dev/
9. Web Apps vs. Websites: Analysis in the React Framework - Evozon, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.evozon.com/web-applications-vs-websites-a-comprehensive-analysis-in-the-react-framework/
10. Best Practices for Testing in ReactJS Development | HDWEBSOFT, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.hdwebsoft.com/blog/best-practices-for-testing-in-react-js-development.html
11. React Native vs. Native: Pros and Cons Explained -

Deftsoft, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://deftsoft.com/blog/react-native-vs-native-pros-and-cons-explained/
12. REACT: Representation Extraction And Controllable Tuning to Overcome Overfitting in LLM Knowledge Editing - arXiv, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://arxiv.org/html/2505.18933v1
13. Comprehensive Guide to ReAct Prompting and ReAct based ..., เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.mercity.ai/blog-post/react-prompting-and-react-based-agentic-systems
14. ReAct Prompting: How We Prompt for High-Quality Results from LLMs | Chatbots & Summarization | Width.ai, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.width.ai/post/react-prompting
15. ReAct agents vs function calling agents - LeewayHertz, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.leewayhertz.com/react-agents-vs-function-calling-agents/
16. ReACT Agent Model - Klu.ai, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://klu.ai/glossary/react-agent-model
17. [2503.23415] An Analysis of Decoding Methods for LLM-based Agents for Faithful Multi-Hop Question Answering - arXiv, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://arxiv.org/abs/2503.23415
18. LLM agents: The ultimate guide 2025 | SuperAnnotate, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.superannotate.com/blog/llm-agents
19. Agentic AI for Intent-Based Industrial Automation This work was partially supported by Coordenação de Aperfeiçoamento de Pessoal de Nível Superior - Brasil (CAPES) – Financial Code 001, by Fundação de Amparo à Pesquisa do Estado de São Paulo (FAPESP) - grant #2020/09838-0, and the Conselho Nacional de Desenvolvimento Científico e

Tecnológico (CNPq) - arXiv, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://arxiv.org/html/2506.04980v1
20. www.ibm.com, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.ibm.com/think/topics/react-agent#:~:text=A%20ReAct%20agent%20is%20an,decision%2Dmaking%20in%20agentic%20workflows.
21. Comparing Chain-of-Thought (CoT) and Tree-of-Thought (ToT) Reasoning Models in AI Agents - MonsterAPI, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://blog.monsterapi.ai/chain-of-thought-cot-and-tree-of-thought-tot-reasoning-models-in-ai-agents/
22. Tree of Thoughts (ToT) - Prompt Engineering Guide, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.promptingguide.ai/techniques/tot
23. Mastering Advanced Prompting: Tree of Thought, ReAct, and Chain of Thought - Techno Billion AI, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.technobillion.ai/post/mastering-advanced-prompting-tree-of-thought-react-and-chain-of-thought
24. LLM Agents: How They Work and Where They Go Wrong - Holistic AI, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.holisticai.com/blog/llm-agents-use-cases-risks
25. Vibe Coding vs. Agentic Coding: Fundamentals and Practical Implications of Agentic AI - arXiv, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://arxiv.org/pdf/2505.19443
26. Top LLM Trends 2025: What's the Future of LLMs - Turing, เข้าถึงเมื่อ มิถุนายน 13, 2025 https://www.turing.com/resources/top-llm-trends